Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:06'da başlar. Kaydın başında görüntü yok (kamera kapalı ekranı).
  - Ders 01:02'de 10 dakikalık bir arayla ikiye bölündü. Aynı günün uygulaması ayrı kayıttadır (ders 10).
- Tekrar: association, dependency, sequence diagram
- Two-way association ve bu ilişkinin riskleri
- Aggregation (parça-bütün, gevşek ilişki)
- Composition (parça-bütün, sıkı ilişki)
  - `Gallery` (galeri) ve `Car` örneği: dizi ile çokluk, arama, silme, satma.
  - Metot aşırı yüklemesi (method overloading): iki farklı `searchCar` metodu.
- Inheritance (kalıtım)
  - Terimler, `protected`, override, `final`, kalıtım ağacı.
  - Polymorphism ve dynamic binding.
  - `extends` ve `super` ile `Person`, `Employee`, `Manager` örneği.
  - Overriding ve overloading farkı.
  - Kalıtımın yanlış kullanımı: GUI düğmesi ve renkler, hasta ve doktor örneği.
  - Tüm sınıfların üst sınıfı `java.lang.Object`, `@Override` ve `toString()`.

Hocanın Özellikle Vurguladığı Kısımlar

- Yalnızca izleyerek nesneye yönelik programlama öğrenilmez. Kod yazmak gerekir.
- Kodunuzu derleye derleye ilerleyin. Derlenmeyen proje export edildiğinde sınıflar boş metotlarla gelebilir.
- Sequence diagram önemli. Bir senaryodan sequence diagram çizebilmelisiniz.
- Two-way association'da iki yönü de atamayı unutmayın.
  - Atama yapılmazsa referans `null` kalır ve `NullPointerException` alınır.
- Aggregation ile composition arasındaki fark senaryodan çıkarılır.
  - Bütün silindiğinde parçalar da yok oluyorsa composition, yaşamaya devam ediyorsa aggregation.
- Kalıtımı her yerde kullanmayın.
  - Önce "is-a" (dır/dir) kontrolü yapın. "Doktor bir hasta mıdır?" Hayır. O zaman kalıtım yoktur.
  - Nesne yönelimli tasarımda association, aggregation ve composition, kalıtımdan daha sık kullanılır.
- Yazılımı değiştirmek, sıfırdan yazmaktan daha zor geliyorsa tasarım kötü yapılmıştır.

Kısa Tekrar Notları

- **Nesne düzeyinde ilişkiler:** association, dependency, aggregation, composition.
- **Sınıf düzeyinde ilişki:** inheritance.
- **Aggregation:** Boş (beyaz) elmas. Parça-bütün ilişkisi ama gevşek. Parçalar bütünden bağımsız yaşayabilir. Örnek: otobüs hattı ve duraklar.
- **Composition:** Dolu (siyah) elmas. Sıkı parça-bütün ilişkisi. Parça aynı anda yalnızca bir bütüne ait olur. Parça nesneler genellikle bütünün içinde `new` ile oluşturulur. Galeri örneğinde `Car` nesneleri dışarıda oluşturuldu; ders sonrası notlara bakın.
- Kardinalite yazılmamışsa 1 kabul edilir.

- **Inheritance gösterimi:** Boş üçgen ok. Ok alt sınıftan üst sınıfa doğrudur.
- **Terimler:** Üst sınıf = parent = superclass = base class. Alt sınıf = child = subclass = derived class.
- **`protected`:** Alt sınıflardan erişilebilir. İlişkisiz başka paketteki sınıflardan erişilemez.
- **Override:** Üst sınıftaki metodu alt sınıfta aynı imzayla yeniden tanımlamak. Yalnızca kalıtımda olur.
- **Overload:** Aynı adlı, farklı parametreli metotlar. Kalıtımla ilgisi yoktur.
- **`final` metot:** Override edilemez. **`final` sınıf:** Kalıtım alınamaz (C#'ta `sealed`).
- **Polymorphism:** Üst sınıf tipinde referanslarla tutulan farklı alt sınıf nesnelerinde, çalışma zamanında doğru metodun çağrılması (dynamic binding).
- **`super(...)`:** Üst sınıfın yapıcısını çağırır. Hoca ilk satırda olması gerektiğini söyledi; sürüm farkı için ders sonrası notlara bakın.
- **`super.metot()`:** Üst sınıftaki metodun override edilmeden önceki hâlini çağırır.
- Java'da bir sınıf yalnızca **bir** sınıftan kalıtım alabilir. Çoklu kalıtım için interface kullanılır.
- Tüm sınıfların örtük üst sınıfı `java.lang.Object`'tir.

Detaylı Açıklamalar

Two-way association

- `Person` sınıfı `Car` referansı, `Car` sınıfı da `owner` adında `Person` referansı tutar.
- One-way'den two-way'e geçerken `Car` sınıfı değişti. `Person` aynı kaldı. Bu yüzden örnek ayrı bir pakete konmuş.
- Kardinalite iki tarafta da `0..1`: Kişinin arabası olmayabilir, arabanın sahibi olmayabilir.
- `Car` sınıfındaki `introduceSelf()` metodu, sahip varsa sahibin bilgisini de yazar. Bunun için `owner != null` kontrolü gerekir.
- `main` içinde iki yön de atanmalıdır (plaka örnek olarak yazıldı):

```
Person octay = new Person("Oktay Sinanoğlu");
Car rover = new Car("34 ABC 34");
octay.setCar(rover);
rover.setOwner(octay);
```

- Bu iki satırdan biri unutulursa ilgili referans `null` kalır.
- Derste tartışılan durum: `c2.setOwner(p1)` sonra `c2.setOwner(p2)` çağrılırsa araba el değiştirmiş olur. `setOwner` tek parametre aldığı için arabanın aynı anda iki sahibi olamaz. Ama `p1` hâlâ `c2`'yi gösteriyor olabilir. Yani iki yön tutarsızlaşabilir.
- Çözüm önerisi: `set` metotlarında karşı tarafı da kontrol etmek ve güncellemek.
  - Örnek: `setCar` içinde, gelen arabanın sahibi `this` değilse arabanın sahibini `this` yap.
  - Slayttaki çözümde `Car` yapıcısı sahibi parametre olarak alıp `setOwner`'a gönderiyor. Böylece nesne oluşurken iki yön birlikte kuruluyor.
- One-way association daha güvenlidir. Karşı taraftan karışma olmaz.

Aggregation

- Bütün-parça (whole-part) ilişkisidir. Association'dan kavramsal olarak daha güçlüdür. Kodlamada fark yoktur.
- Örnek: Bir otobüs hattında (bus route) en az bir durak (stop) vardır.
- Bütün ve parça birbirinden bağımsız yaşayabilir.
  - Hocanın örneği: 41AT hattını silerseniz Eyüp durağı silinmemelidir. O duraktan başka hatlar da geçer.
  - Öğrenci derse kayıtlıdır. Öğrenci dersi bırakınca ders silinmez.
- Parça nesneler dışarıda oluşturulur, sonra bütüne atanır (set metodu veya yapıcı parametresi ile). Bütünün içinde `new` ile oluşturulmaz.

Composition

- Sıkı bütün-parça ilişkisidir. Parça aynı anda yalnızca bir bütüne aittir.
- Bütün silinince parçalar da yok olur. Hocanın örneği: İnsan ölünce organları da yok olur. Kütüphane yıkılınca raflar yok olur ama kitaplar yok olmak zorunda değildir.

- Slayttaki takım-oyuncu örneğini hoca iyi bulmadı. Oyuncu transfer olabilir, yani takımla birlikte yok olmaz.
- Çokluk (multiplicity) dizi veya veri yapıları (liste, kuyruk, yığın, bağlı liste, ağaç, graf) ile kodlanır.

Galeri örneği (composition)

- `Person` ile `Car` arasında two-way association var.
- `Gallery` ile `Car` arasında composition var. Galerideki arabalar bir dizi ile tutulur. Galerinin yapıcısında dizi `new` ile oluşturulur.
- `Gallery` ile `Person` (galeri sahibi) arasında one-way association var. Galeri insan satmaz, aralarında parça-bütün ilişkisi yoktur.

Galeri sınıfının alanları ve yapıcısı (derste anlatılana göre yeniden yazıldı; slayttaki adlar farklı olabilir):

```
public class Gallery {
    public final static int MAX_CARS = 30;
    private String name;
    private Car[] cars;
    private int carCount;
    private Person owner;

    public Gallery(String name, Person owner) {
        this.name = name;
        this.owner = owner;
        this.carCount = 0;
        this.cars = new Car[MAX_CARS];   // composition: dizi galerinin içinde oluşturulur
    }
}
```

- `cars.length` dizinin kapasitesini verir, dolu eleman sayısını vermez. Dolu eleman sayısı için `carCount` gerekir.
- `public final static int` sabiti değiştirilemez ve tüm nesnelerde aynıdır.

Araba ekleme:

- Önce araba galeride var mı diye bakılır. Yoksa ve galeri dolu değilse eklenir, `carCount` bir artar, `true` döner. Aksi hâlde `false` döner.

Arama (slayttan):

```
public boolean searchCar(Car aCar) {
    for (Car car : cars)
        if (car == aCar)
            return true;
    return false;
}

public Car searchCar(String plate) {
    for (int i = 0; i < carCount; i++)
        if (cars[i].getPlate().compareTo(plate) == 0)
            return cars[i];
    return null;
}
```

- İlk metot referans (adres) karşılaştırır. İkincisi plakayı `compareTo` ile karşılaştırır.
- İki metodun adı aynı, parametreleri farklı. Bu **method overloading**'dir. Yalnızca dönüş tipi farklı olan iki metot aşırı yüklenemez, derleme hatası verir.

Silme ve satma (slayttan):

```
public Car removeCar(String plate) {
    for (int i = 0; i < carCount; i++) {
        if (cars[i].getPlate().compareTo(plate) == 0) {
```

```
            Car theCar = cars[i];
            for (int j = i; j < carCount; j++)
                cars[j] = cars[j + 1];
            cars[carCount - 1] = null; carCount--;
            return theCar;
        }
    }
    return null;
}

public boolean sellCar(String plate, Person newOwner) {
    Car soldCar = removeCar(plate);
    if (soldCar != null) {
        soldCar.setOwner(newOwner);
        return true;
    }
    return false;
}
```

- Silinen elemandan sonrakiler bir sola kaydırılır (shift). Son eleman tekrar etmesin diye son konum `null` yapılır ve `carCount` azaltılır.
- Silinen araba, kullanıcıya bilgi vermek için döndürülür.
- Test sınıfında `new Gallery("...", new Person("..."))` gibi, referansa atanmadan oluşturulan bir nesne doğrudan parametre olarak gönderilebilir.

Inheritance (kalıtım)

- Sınıf düzeyinde bir ilişkidir. Çalışma zamanında değiştirilemez.
- "is-a" ilişkisidir: Jeep bir arabadır, lazer yazıcı bir yazıcıdır.
- Kurallar:
  - Alt sınıf, üst sınıftaki bir üyeyi reddedemez (silemez).
  - Kalıtılan metotların gövdesi alt sınıfta değiştirilebilir. Buna **overriding** denir.
  - Alt sınıfa yeni üyeler eklenebilir.
  - Bir alt sınıf başka sınıfların üst sınıfı olabilir. Ör. `Employee`, `Person`'ın alt sınıfı; `Director` ve `PartTime`'ın üst sınıfıdır.
  - Bu yapıya kalıtım ağacı denir. Bu çok seviyeli (multi-level) kalıtımdır, çoklu (multiple) kalıtım değildir.
  - Ağacı çok derin yapmayın. En üst sınıftaki değişiklik bütün alt sınıfları etkiler.
- Override örneği:
  - `Person` sınıfında `introduceYourself()` "Ben insanım" yazar.
  - `Retired` sınıfında aynı metot "Ben emekliyim" yazacak şekilde yeniden tanımlanır.
  - Override etmeyen `Employee`, `Student` gibi sınıflar `Person`'daki hâli kullanmaya devam eder.
  - Hocanın örneği İstanbulkart: Herkes kişidir ama öğrenci, öğretmen ve emekli için ücret farklı düşer.
- **Polymorphism:**
  - Genel kavram üst sınıfta durur (ör. `Printer` sınıfında `print()`). Nasıl yapılacağı alt sınıflarda tanımlanır (nokta vuruşlu, lazer, mürekkep püskürtmeli).
  - Üst sınıf, daha az yetenekli ve genel bir sürümdür. Alt sınıf daha özel ve daha yeteneklidir.
  - **Substitutability:** Üst sınıfın beklendiği her yerde alt sınıf kullanılabilir.

```
Person[] p = new Person[10];
p[0] = new Student(...);
p[1] = new Retired(...);
p[2] = new Employee(...);
for (Person kisi : p)
    if (kisi != null)
        kisi.introduceYourself();   // her nesnenin kendi sürümü çalışır
```

- Doğru metodun çalışma zamanında seçilmesine **dynamic binding** denir.

`extends` ve `super`: `Person`, `Employee`, `Manager`

Slayttaki kod (kareden):

```
package nyp05;
public class Person {
    private String name;
    public Person(String name) { this.name = name; }
    public String getName() { return name; }
}

public class Employee extends Person {
    private int salary;
    public Employee(String name, int salary) {
        super(name);
        this.salary = salary;
    }
    public int getSalary() { return salary; }
    public void setSalary(int salary) { this.salary = salary; }
}
```

- `name` private olduğu için `Employee` ona doğrudan erişemez. Yapıcıda `super(name)` ile, metotlarda `getName()` ile dolaylı erişir.
- Hoca `super(...)` çağrısının yapıcının **ilk satırında** olması gerektiğini söyledi. Bu kural dersin yapıldığı sürümler için geçerlidir; JDK 25 için ders sonrası notlara bakın.
- Hoca uyardı: Bu örnekte `super(name)` veya `super(salary)` tek parametreyle yazılırsa hata alınır, çünkü `Employee` sınıfında böyle bir yapıcı yoktur (02:01:12).
- `super(...)` yazılmazsa derleyici, üst sınıfın parametresiz yapıcısını (`super()`) örtük olarak çağırır.
- `Manager extends Employee`:
  - Ek alan `bonus`. Yapıcıda `super(name, salary)` çağrılır, `bonus = 0` atanır.
  - `getSalary()` override edilir: `return super.getSalary() + bonus;`
  - `super.salary` yazılamaz, çünkü `salary` private. `super.super` diye bir kullanım da yoktur.
- `main` içinde `Employee[] staff = new Employee[3];` dizisine bir `Manager` ve iki `Employee` konur. `for (Employee e : staff)` döngüsüyle `getName()` ve `getSalary()` yazdırılır. `Manager` için bonuslu maaş çıkar.
- Alıştırma: Bu `main` için sequence diagram'ı döngü kutusuyla (loop) çizin.
- Overloading örneği: `Manager` sınıfına farklı parametreli ikinci bir yapıcı eklemek.

Kalıtımın yanlış kullanımı

- **GUI düğmesi ve renkler:** "Kırmızı bir GUI düğmesidir" denemez. Renk, düğmenin bir özelliğidir.
  - Doğru model: `GuiButton` sınıfında `RGB` tipinde bir renk alanı. Aralarında association vardır.
  - Parça-bütün ilişkisi olmadığı için aggregation veya composition değildir.
- **Hasta ve doktor:** İlk cümleler okununca "doktorda hastanın tüm alanları var, doktor hastadan kalıtım alsın" denebilir.
  - Ama doktor bir hasta değildir. Ayrıca tedaviler (treatments) hastaya aittir, doktora geçmemelidir.
  - Doğru model: Ortak alanlar (ad, T.C. kimlik no, doğum tarihi, telefon) `Person` üst sınıfında. `Patient` ve `Doctor` bu sınıftan kalıtım alır.
  - `Treatment` ayrı bir sınıftır. Bir hastanın birden çok tedavisi olabilir (association). `Treatment`, `Doctor`'ı kullanır (dependency).
- **Java'da çoklu kalıtım yok:** `class Student extends Person, Worker` yazılamaz. Bunun için interface kullanılır.

`Object` sınıfı ve `toString()`

- `java.lang.Object`, tüm sınıfların örtük üst sınıfıdır.

- Bu sınıftaki metotlar override edilebilir. `@Override` notasyonu (annotation) zorunlu değildir ama override edildiğini açıkça gösterir.
- `toString()` override edilirse nesne doğrudan `System.out.println(nesne)` ile yazdırılabilir.
- Dönüş tipi değiştirilemez. `toString()` her zaman `String` döner.

Sonraki ders

- Uygulama oturumunda kalıtım, polymorphism ve overloading üzerine örnek yapılacak.
- Gelecek hafta: abstract sınıflar, çoklu kalıtım tartışması, interface ve enumeration (yaklaşık 134. sayfaya kadar). Sonra Java'daki veri yapılarına geçilecek.

Ders sonrası notlar

- **Slayttaki `removeCar` kodunda bir hata var.** İç döngü `for (int j = i; j < carCount; j++) cars[j] = cars[j + 1];` şeklinde. Dizi doluyken (`carCount == cars.length`) son adımda `cars[carCount]` okunur ve `ArrayIndexOutOfBoundsException` alınır. Java ile denendi: 3 elemanlık dolu dizide `Index 3 out of bounds for length 3` hatası çıktı. Doğrusu:

```
for (int j = i; j < carCount - 1; j++)
    cars[j] = cars[j + 1];
cars[carCount - 1] = null;
carCount--;
```

- `searchCar(Car aCar)` içindeki `for (Car car : cars)` döngüsü dizinin tamamını (boş `null` konumlar dahil) gezer. `car == aCar` karşılaştırması `null` ile hata vermez. Ama `searchCar(null)` çağrılırsa ve dizide boş yer varsa yanlışlıkla `true` döner. Döngüyü `carCount`'a kadar kurmak daha doğrudur.
- Hoca `protected` için "kalıtım olmayan başka sınıflar erişemez" dedi. Java'da `protected` üyelere **aynı paketteki** tüm sınıflar da erişebilir. Başka paketteki ilişkisiz sınıflar erişemez.
- Slaytta "Private methods of a super class can be overridden in its subclass" deniyor. Java'da private metotlar kalıtılmaz ve **override edilemez**. Alt sınıfta aynı imzayla bir metot yazılabilir ama bu yeni ve bağımsız bir metottur. Üzerine `@Override` yazılırsa derleme hatası verir. Slaytın devamındaki "alt sınıf orijinal sürüme `super` ile de erişemez" ifadesi doğrudur.
- Hoca, `super` çağrısının ilk satırda olması gerektiğini genel bir kural gibi söyledi. Bu kural yalnızca **yapıcı** içindeki `super(...)` çağrısı içindir. Normal bir metot içinde `super.getSalary()` gibi çağrılar herhangi bir satırda olabilir. Hoca bunu bir sonraki derste (ders 10) kendisi de gösterdi.
- **`super(...)` ve JDK 25:** JDK 25'te `super(...)` öncesinde başka deyim yazılabilir: `int b = a + 1; super(b);` derlendi. `-source 21` ile aynı kod hata veriyor. Hocanın söylediği kural dersin yapıldığı dönem için doğrudur.
- **Composition ölçütü:** Hoca, parçaların bütünün içinde üretildiğini söyledi. Galeri örneğinde `new Car[...]` yalnız `null` referanslardan oluşan diziyi üretir; `Car` nesneleri dışarıda oluşturulup eklenir. Bu örnek ölçütü tam karşılamaz.
- Hoca, "overloading association, aggregation ve composition ile ilgilidir" dedi. Overloading herhangi bir ilişki türüyle ilgili değildir. Aynı sınıftaki aynı adlı, farklı parametreli metotlar demektir. Kalıtımda alt sınıf da üst sınıftaki bir metodu aşırı yükleyebilir.
- `final` metot tanımında hoca sırayı hatırlamadığını söyledi. Geleneksel yazım `public final String introduceSelf()` şeklindedir. `public final` ve `final public` ikisi de derlenir, ama `final` dönüş tipinden sonra gelemez: `public String final m()` derlenmedi (Java ile denendi).
- Hoca "public kullanırsak kalıtıma gerek yok" dedi. Kastedilen, `protected` anahtar kelimesinin bilgiyi yalnızca alt sınıflara açmak için kullanıldığıdır. Kalıtımın amacı yalnızca erişim değildir; ortak davranışı paylaşmak ve polymorphism sağlamaktır.

- Altyazıdaki “Nive/near” `new`, “segmenter” `setCar`, “dinamik binding” dynamic binding, “Isarılayışı” “is-a ilişkisi”, “hez” “has” demektir. “Lazar” lazer yazıcıdır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.